Machine Learning and Deep Learning
Lecture-11

By: Somnath Mazumdar
Assistant Professor
sma.digi@cbs.dk

Spring 2026

Overview

- Autoencoder
- Brief intro to attention/transformers

Autoencoder

Autoencoder

- Autoencoders are:
  - unsupervised (self-supervised learning)
  - capable of randomly generating new data that looks very similar to training data.
  - Variational autoencoders (VAEs) are truly generative.

- ANNs capable of learning dense representations of input data, called latent representations or codings.
- Codings typically have a much lower dimensionality than input data.
- Autoencoders are useful for dimensionality reduction (good for visualization)

- Autoencoders simply learn to copy their inputs to their outputs.

Autoencoder

- Codings are byproducts of autoencoder learning identity function under some constraints.
- An autoencoder looks at inputs, converts them to an efficient internal latent representation, and then outputs something that (hopefully) looks very close to inputs.

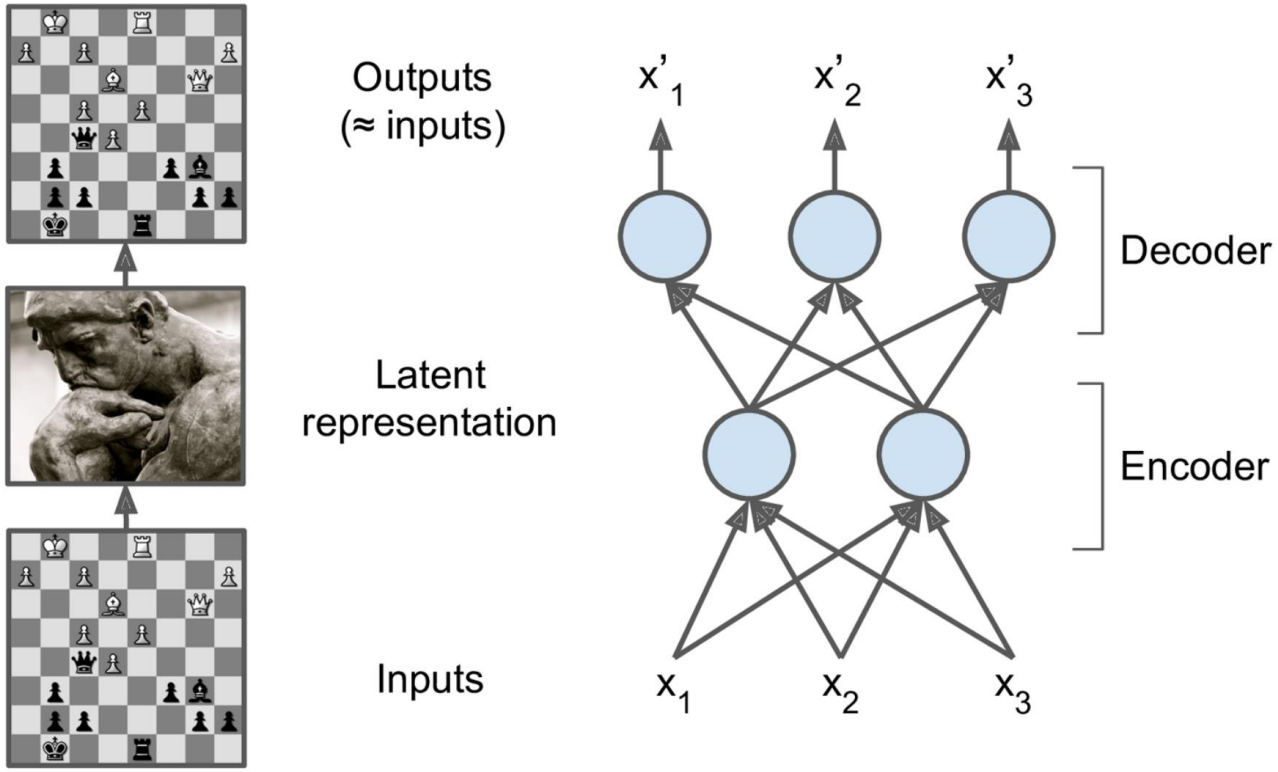
- E.g.,: you can add noise to inputs and train network to recover original inputs.

- Autoencoder composed of two parts:
  - An encoder (or recognition network) that converts inputs to a latent representation.
  - A decoder (or generative network) that converts internal representation to outputs.

Autoencoder

- Autoencoder has same architecture as MLP except number of neurons in output layer must be equal to number of inputs.

- Here is a single hidden layer composed of two neurons (encoder).
- An output layer composed of three neurons (decoder).
- Outputs are often called reconstructions because autoencoder tries to reconstruct inputs



- Latent representation captures the essential structure or features in a lower-dimensional space.
- Encodes meaningful structure.

Autoencoder

- Cost function contains a reconstruction loss that penalizes model when reconstructions are different from inputs.

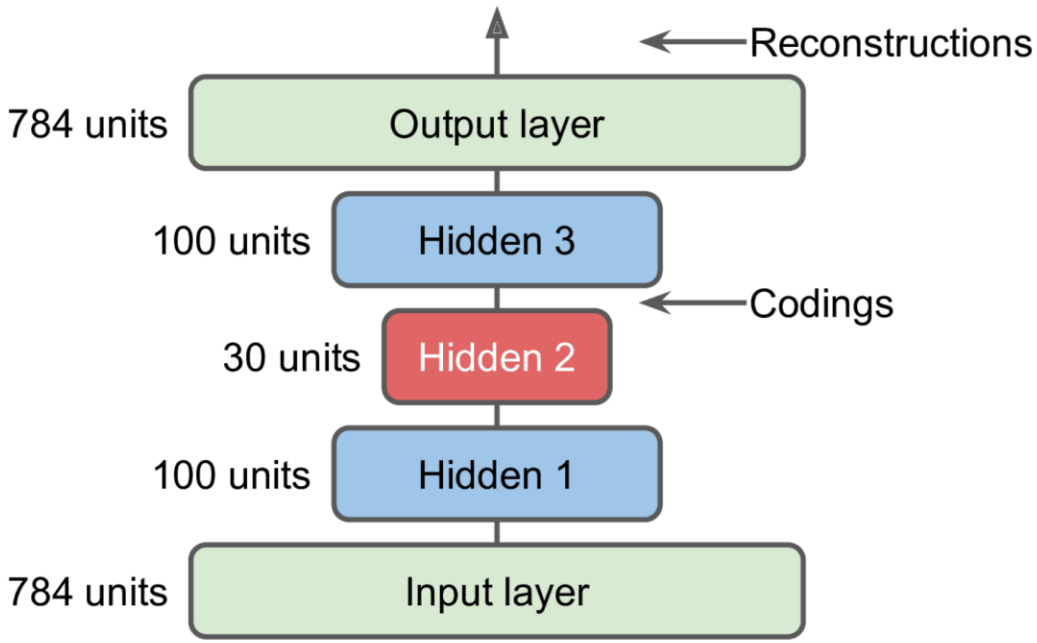
- An undercomplete autoencoder is one where internal representation has a lower dimensionality than input data.
  - An undercomplete autoencoder cannot trivially copy its inputs to codings, and is forced to learn most important features in input data (and drop unimportant ones).

- If autoencoder uses only linear activations and cost function is mean squared error (MSE), then it ends up performing Principal Component Analysis (PCA).

Stacked Autoencoder

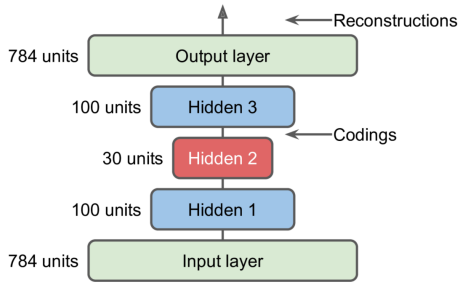
- Stacked autoencoders (or deep autoencoders): Autoencoders can have multiple hidden layers.
  - Advantage: Adding more layers helps autoencoder learn more complex codings.

- An autoencoder for MNIST has 784 inputs, followed by a hidden layer with 100 neurons.
- A central hidden (coding) layer of 30 neurons, then another hidden layer with 100 neurons, and an output layer with 784 neurons



Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurélien Géron, 2019

Stacked Autoencoder



The network is forced to compress information into 30 dimensions, so: z becomes a latent representation. It captures the most important structure of the image

- Stacked autoencoder very much like regular deep MLP.
- compresses 28×28 images into a 30-dimensional latent vector and reconstructs them back.

```
stacked_encoder = keras.models.Sequential([
keras.layers.Flatten(input_shape=[28, 28]),
keras.layers.Dense(100, activation="selu"),
keras.layers.Dense(30, activation="selu"),
])
```

- Flatten: converts a 28×28 image → vector of size 784
- Dense(100): reduces 784 → 100 features
- Dense(30): reduces 100 → 30 features
- [This 30-dimensional vector is the latent representation.]

SELU (Scaled Exponential Linear Unit) designed to make certain feedforward neural networks self-normalizing, meaning their activations tend to remain approximately centered and appropriately scaled across layers during training.

```
stacked_decoder = keras.models.Sequential([
keras.layers.Dense(100, activation="selu", input_shape=[30]),
keras.layers.Dense(28 * 28, activation="sigmoid"),
keras.layers.Reshape([28, 28])
])
```

- Takes latent vector (size 30)
- Expands: 30 → 100 → 784
- Reshapes back to 28×28
- output: reconstructed image

```
stacked_ae = keras.models.Sequential([stacked_encoder, stacked_decoder])
stacked_ae.compile(loss="binary_crossentropy",
optimizer=keras.optimizers.SGD(learning_rate=1.5))
history = stacked_ae.fit(X_train, X_train, epochs=10,
validation_data=[X_valid, X_valid])
```

- Loss: compares original image with reconstructed image
- Binary crossentropy: suitable for pixel values in [0,1]
- SGD: optimization algorithm

Convolution Autoencoder

- If you want to build an autoencoder for images, you need to build a convolutional autoencoder.
  - Encoder is a regular CNN composed of convolutional layers and pooling layers.
  - It reduces spatial dimensionality of inputs (i.e., height and width) while increasing depth (i.e., number of feature maps).
  - Decoder must do reverse (upscale image and reduce its depth back to original dimensions), and for this use transpose convolutional layers.

Recurrent & Denoising Autoencoders

- If you want to build an autoencoder for sequences (such as time series or dimensionality reduction), then RNNs may be better suited than dense networks.
- Recurrent autoencoder:
  - Encoder is typically a sequence-to-vector RNN which compresses input sequence down to a single vector.
  - Decoder is a vector-to-sequence RNN that does reverse.

- Denoising Autoencoders: force autoencoder to learn useful features by adding noise to its inputs, training it to recover original (noise-free).
- Noise can be pure Gaussian noise added to inputs, or it can be randomly switched-off inputs (dropout).
- Autoencoders could also be used for feature extraction*.

*Pascal Vincent et al., "Extracting and Composing Robust Features with Denoising Autoencoders," Proceedings of the 25th International Conference on Machine Learning (2008): 1096–1103.

Attention/Transformers

Why?

- Language processing:
  - A series of techniques to break down and understand text:
- **Parsing**: Analyzes the sentence structure
- **Tokenization**: Splits sentences into individual words (tokens)
- **Stemming**: Reduces words to their root form, ensuring the model treats similar words consistently
- **Entity recognition and relationship extraction**: Identifies and categorizes specific entities
- **Word embeddings**: Creates a numerical representation for each word (a vector), capturing its meaning and connections to other words

- Limitations of traditional models: Limited/short context + Word disambiguation issues + Generalization challenges
- Attention allows language models to consider broader context.

Attention Mechanism

- Transformers use attention mechanisms.
- Attention was introduced in 2014-2015, with the Transformer architecture proposed in 2017.
- Transformers (and attention) have become the foundation for LLMs.
- Attention assigns weights to each word based on its relevance to the current task, which enables the model to capture long-range dependencies.

Attention Mechanism

- Attention enables models to understand nuances and ambiguities in language.
- Benefits:
  - Dynamic weighting: Attention allows models to dynamically adjust the importance of words based on their relevance to the current context
  - Long-range dependencies: Enables capturing relationships between words situated at long distances in the sequence
  - Contextual understanding: Provides contextualized representations that help resolve ambiguities and make models adaptable to various downstream tasks
- Applications:
  - Machine translation (e.g., Google Translate)
  - Text summarization
  - Sentiment analysis
  - Content generation

Self-attention

- Self-attention enables a model to attend to different positions within its input sequence to compute a contextualized representation.
- Each token can weigh the importance of every other token in the sequence.

- **Key Components:**
  - **Query (Q):** A vector representing what the model is "asking" about a specific position
  - **Key (K):** A vector representing what information each position "offers"
  - **Value (V):** A vector containing the actual content/information at each position

- **Computation:** Attention scores are calculated using a scaled dot-product between query and key vectors, followed by a softmax to obtain attention weights, which are then used to compute a weighted sum of values.

Multi-head attention

- Extends self-attention by running multiple attention operations in parallel, each learning different representation subspaces.

- **Mechanism**:
  - Splits Q, K, V into h heads
  - Performs scaled dot-product attention independently on each head
  - Concatenates outputs and applies a final linear transformation

- **Benefits**:
  - Captures diverse linguistic patterns simultaneously
  - Provides finer-grained contextual understanding
  - Increases model robustness and expressivity

Self-attention and multi-head attention

- Challenges:
  - Computationally expensive for long sequences
  - May overfit noisy or irrelevant information in the input sequence, leading to suboptimal performance on unseen data
  - Requires significant memory and computational resources, making it difficult to scale to larger models and datasets

Feature	Attention-Based Models	Traditional NLP Models
Word Representation	Context-aware embedding vectors (dynamically weighted based on attention scores)	Static embedding vectors (single vector per word, no context considered)
Focus	Considers surrounding words for meaning (looking at the broader context)	Treats each word independently
Strengths	Captures long-range dependencies, resolves ambiguity, understands nuances	Simpler, computationally cheaper
Weaknesses	Can be computationally expensive	Limited ability to understand complex language, struggles with context
Underlying Mechanism	Encoder-decoder networks with attention (various architectures)	Techniques like parsing, stemming, named entity recognition, word embeddings

Transformers

Transformers

- Transformers are NNs that use self-attention mechanisms to process sequential data in parallel.
- State-of-the-art architecture for many NLP tasks.
- Are considered an evolution of the encoder-decoder architecture.
- Core components include input embeddings, positional encoding, multi-head self-attention, feed-forward networks, residual connections, and layer normalization.

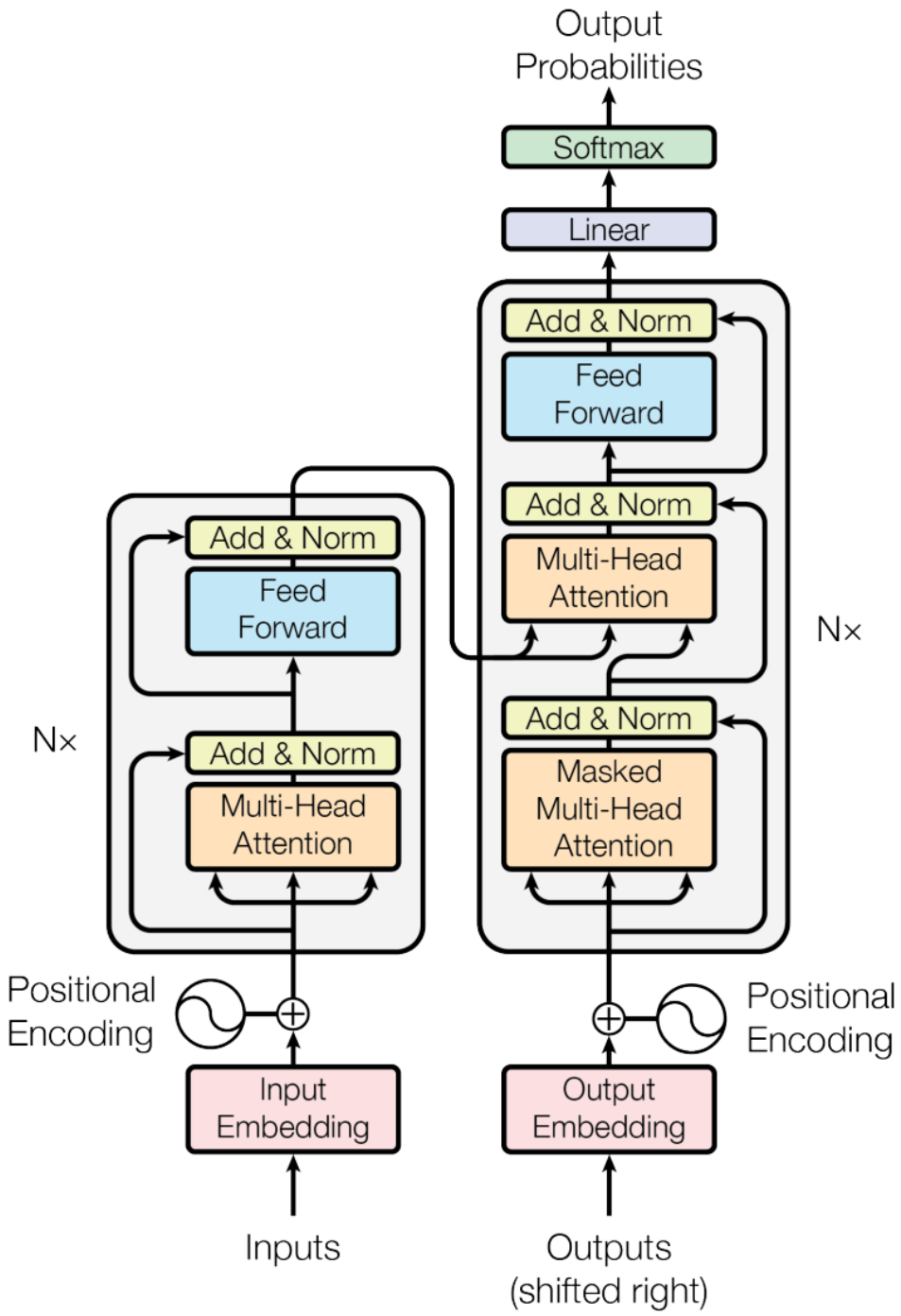
- Before Transformers, RNNs were the preferred approach for processing sequential data.
- Transformers were inspired in part by the encoder-decoder architecture used in RNN-based sequence-to-sequence models.
- In addition to improving performance over RNN-based models, Transformers introduced a new architecture for sequence modeling.
- From LSTMs to Transformers in NLP, attention mechanisms brought two major improvements:
    - They allow models to focus on relevant words, regardless of distance in the sequence.
    - They improve computational efficiency by enabling parallel processing.
- For these reasons, Transformers became a natural advancement over RNNs in many NLP applications.

Architecture

- Input Tokens → Embeddings → Positional Encoding → N×Encoder Layers → Nx Decoder Layers → Linear → Softmax → Output Sequence

- Tokenization
- Input Embeddings
- Position Encodings
- Query, Key, & Value
- Attention
- Self Attention
- Multi-Head Attention
- Feed Forward
- Add & Norm
- Encoders

- Masked Attention
- Encoder Decoder Attention
- Linear
- Softmax
- Decoders
- Encoder-Decoder Models



https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

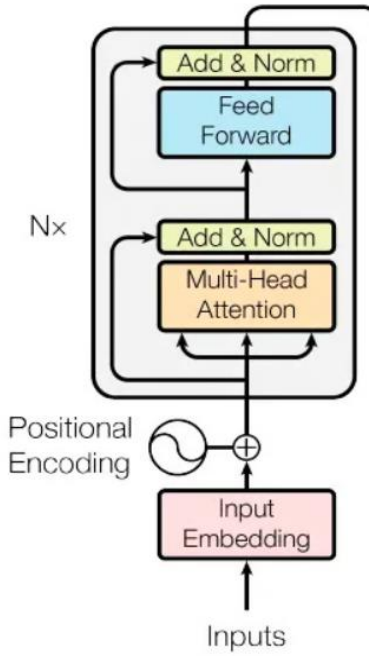
Architecture

- Core Properties
  1. Encoder-decoder architecture
  2. Self-attention replaces recurrence
  3. Parallelizable computation
  4. Deep stacked architecture
  5. Residual connections with layer normalization

Encoder Architecture

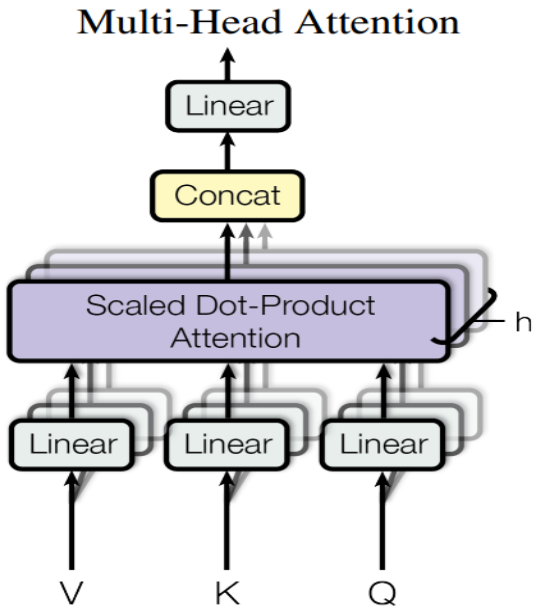
- The encoder transforms an input sequence into a contextualized representation.
- **Input embeddings**: Each token is converted into a dense vector of fixed dimension (e.g., 512).
  - An embedding layer converts tokens into dense vectors before they are passed to the encoder stack.
- **Positional encoding**: Explicitly injects sequence-order information into the embeddings.
  - Positional encodings are added to the input embeddings, use sine and cosine functions of different frequencies, and support generalization to sequences of arbitrary length.
  - They create position-dependent vectors in the range [-1, 1].

- Encoder Layer Structure (stacked N times)
  - Multi-head self-attention
  - Add & Layer Normalization
  - Position-wise Feed-Forward Network
  - Add & Layer Normalization



Multi-Head Self-Attention (Encoder)

- Multi-Head Self-Attention (Encoder) Self-attention allows each token to attend to every other token in the sequence.
  - For each token, three vectors are computed through learned linear projections: Query (Q), Key (K), and Value (V).



- Scaled Dot-Product Attention
  - Step 1: Compute attention scores
  - Step 2: Scale the scores to stabilize gradients
  - Step 3: Apply Softmax
  - Step 4: Compute a weighted sum of the value vectors

- Multi-Head Attention
  - Instead of applying attention once, the model splits Q, K, and V into "h" heads.
  - It performs attention in parallel across all heads.
  - The outputs are concatenated and passed through a final linear transformation.

Multi-Head Self-Attention

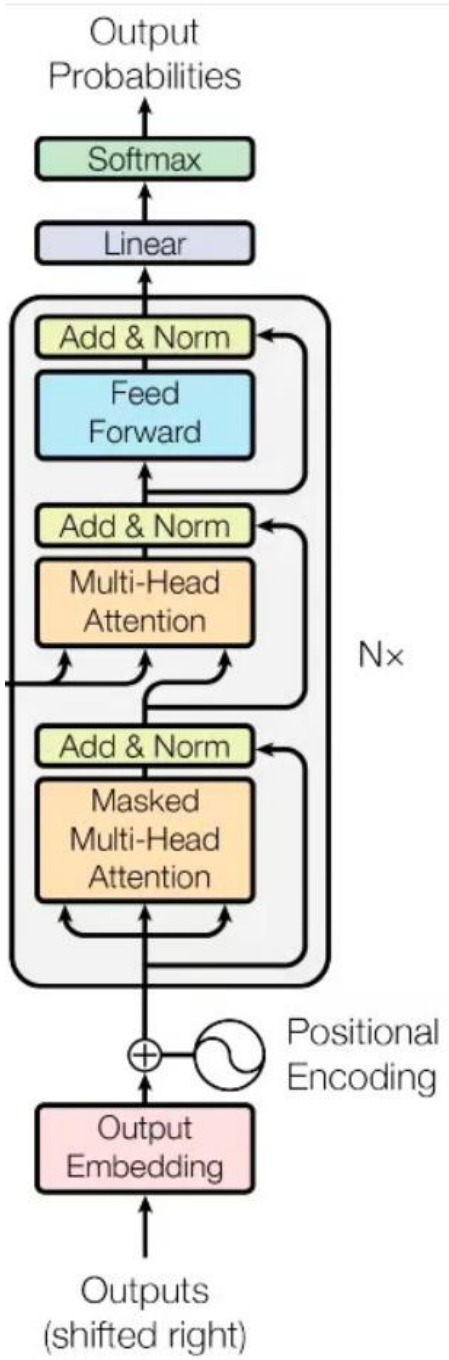
- Residual Connections and Layer Normalization
  - Help mitigate vanishing gradients
  - Enable deep stacking
  - Stabilize training
- Feed-Forward Network (FFN)
  - Each encoder layer contains a position-wise FFN
  - It consists of two linear layers with a ReLU activation
  - It is applied independently at each position
- Encoder Output
  - After NN layers, the encoder outputs a sequence of contextualized vectors
  - These vectors represent the full sequence and serve as input to the decoder

- Note: in many Transformer variants, the FFN may use Gaussian Error Linear Unit (GELU) instead of ReLU.

Multi-Head Self-Attention

- Decoder Architecture:
  - Generates output sequences autoregressively
  - Uses masked self-attention
  - Uses encoder-decoder (cross) attention
  - Generates tokens sequentially

- Each decoder layer contains:
  - Masked multi-head self-attention
  - Encoder-decoder multi-head attention
  - Feed-forward network
  - Residual connections and layer normalization after each sublayer
- Masked self-attention prevents access to future tokens.



Multi-Head Self-Attention

- **Encoder-Decoder (Cross) Attention**
  - Queries come from the decoder
  - Keys and values come from the encoder output
  - This allows the decoder to align generated tokens with relevant input positions and integrate source and target information

- **Output Projection**
  - The final decoder output passes through a linear layer whose output dimension equals the vocabulary size
  - A Softmax layer is then applied to produce token probabilities
  - The highest-probability token is selected at each step
  - Decoding continues until an end token is predicted

- In practice, decoding does not always strictly choose the highest-probability token.

Transformers

- Transformer Model Examples:
  - BERT: A pre-trained language model introduced by Google, known for bidirectional training
  - GPT and ChatGPT: Transformer-based models developed by OpenAI

- **Modern Transformer Variants**
  - Vision Transformers (ViT): Apply self-attention to images by dividing them into patches and treating each patch as a token
  - Multimodal Transformers: Process multiple input types, such as text, images, audio, and video, within a single architecture

- **Drawbacks and Limitations**
  - Quadratic complexity: Self-attention scales as $O(n^2)$ with sequence length, making very long contexts computationally expensive
  - Memory requirements: Large models require significant GPU memory
  - Training data requirements: Require massive datasets
  - Lack of explicit reasoning: Transformers may hallucinate facts and do not perform symbolic reasoning explicitly

Summary

- Autoencoders learn to copy their inputs to their outputs by converting the inputs into an efficient latent representation and then reconstructing something close to the original inputs.
- **Structure**: An autoencoder consists of two main parts: an encoder, which converts inputs into a latent representation, and a decoder, which reconstructs the outputs from that representation.
  - The cost function includes a reconstruction loss that penalizes differences between the input and the reconstruction.
  - An autoencoder has a structure similar to that of a multilayer perceptron (MLP), except that the output layer must have the same number of neurons as the input.
  - A recurrent autoencoder typically uses a sequence-to-vector RNN as the encoder and a vector-to-sequence RNN as the decoder.
- **Application**: Autoencoders are useful for dimensionality reduction, especially for visualization

Summary

- Neural networks using self-attention to process sequential data in parallel; state-of-the-art for many NLP tasks
- **Architecture:** Encoder-decoder structure with multi-head self-attention, positional encoding, feed-forward networks, residual connections, and layer normalization

- **Key Innovation:** Self-attention mechanism enables:
  - Parallel computation
  - Long-range dependency capture
  - Dynamic contextual understanding
- **Applications:** Language models (BERT, GPT), machine translation, text summarization, vision transformers, multimodal models

- **Advantages:** Faster training than RNNs, better performance on long sequences, highly parallelizable

- **Limitations:** $O(n^2)$ complexity, high memory requirements, massive data needs, may hallucinate facts